

Reviewer Pack — Minimal Rank of Quasi-Group Representations vs Monotone k-CL...

Entry f209b3d60500 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 04:18:02 UTC

Minimal Rank of Quasi-Group Representations vs Monotone k-CLIQUE Circuit Size

Entry ID: f209b3d60500 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 04:18:02 UTC

1. Conjecture

Field A (mathematical branch): Quasi-group Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

[‘The minimal rank of the quasi-group representation associated with a monotone circuit computing k-CLIQUE is $\Omega(n^k)$, where n is the number of variables in the circuit.’]

Rationale (proposer’s reasoning):

[‘Quasi-groups provide a framework for describing symmetries, and their representations can encode non-trivial combinatorial structures. If this conjecture holds, it would imply that k-CLIQUE requires a large amount of symmetry information, suggesting inherent complexity in the problem.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4eff765069c6075d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of the quasi-group representation for monotone k-CLIQUE circuits is $\Omega(n^k)$, where n is the number of variables.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	1.00	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quasi-group theory" AND "monotone k-CLIQUE circuit complexity" - "minimal rank" AND "quasi-group representation" AND "circuit size" - " $\Omega(n^k)$ " AND "quasi-group representation" AND "monotone circuit"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0

    for i in range(cols):
        max_row = None
        for j in range(i, rows):
            if matrix[j][i] != 0:
                max_row = j
                break

        if max_row is None:
            continue

        # Swap rows
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below pivot
        for j in range(i + 1, rows):
            factor = -matrix[j][i] / matrix[i][i]
            for k in range(i, cols):
                matrix[j][k] += factor * matrix[i][k]

        rank += 1

    return rank

def minimal_rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    sub_matrix = [[matrix[i][j] for j in range(cols)] for i in range(rows)]
```

```

    # Perform Gaussian elimination
    rank = gaussian_elimination(sub_matrix)

    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    k = 5

    # Generate a monotone circuit for k-CLIQUE
    # This is a simplified representation and not a real monotone circuit
    circuit = []
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                circuit.append((i, j))

    # Map circuit gates to group operations (simplified)
    matrix = [[0] * n for _ in range(n)]
    for gate in circuit:
        i, j = gate
        matrix[i][j] = 1

    rank = minimal_rank(matrix)

    return {
        "metric_name": "minimal_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank >= n**k,
        "counterexample": "" if rank >= n**k else f"Rank {rank} < {n**k}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    std_rank = math.sqrt(sum((result["metric_value"] - mean_rank)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = results[first_failing_seed]["counterexample"]

```

```
print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds':  
RESULT: FALSIFIED counterexample="Rank 0 < 102400000" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The conjecture has been falsified with a single instance, which is insufficient to confirm the statement. The failure mode ‘n too small’ is evident as only one instance was tested. Additionally, the counterexample provided (Rank 0) suggests that the test may not be measuring what the statement claims.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The conjecture has been falsified by a counterexample where the rank of the quasi-group representation associated with a monotone circuit computing k- | next: Further investigation is needed to understand why the test produced a rank of 0 and whether this is an isolated case or indicative of a broader issue. It may be necessary to re-evaluate the testing conditions and methodology.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	9813
2	preregistration	ollama_remote	glm4:latest	0	0	5755
3	novelty	ollama_remote	glm4:latest	0	0	4742
4	novelty	ollama_remote	glm4:latest	0	0	5418
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15011
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10488

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10207
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9477
9	critic	ollama_remote	glm4:latest	0	0	11969
10	judge	ollama_remote	glm4:latest	0	0	6195

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89075 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/f209b3d60500.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f209b3d60500.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f209b3d60500.tar.gz (if generated)